RAYMOND SEKIKUBO

Part 1: QA Strategy & Release Readiness

Mentors must be able to register a youth participant using their phone number, full name, age (must be between 15–24), and district. 2. If internet connectivity is lost, registration data collected via the web portal must queue locally and automatically sync when online. 3. Youth who register via USSD must receive an immediate SMS confirmation with a unique 6-digit Participant ID.

1. **Definition of Done Checklist:**

- A mentor can successfully register a participant with their phone number, full name, age and district
    - Separate fields for name, and not 1 field which could capture 1 name and not full names
- Age accepted from 16 to 23 years, and rejected below or above the specified range.
    - Age accepted is only numeric and no text, empty and special characters accepted.
- Phone numbers are validated and normalised to one format, Check for duplicates and flag at submission.
- District comes from a controlled list, not free text.
- Web registrations made offline are queued locally, shown to the mentor as "pending sync", and sent automatically when connectivity returns, without a manual retry (AC2).
- A USSD registration triggers an SMS with a unique 6-digit Participant ID within the agreed time

2. **Bug Triaging Scenario:**

BUG	Severity	Priority	Impact
B: Duplicate records after reconnect	Critical	P1	Blocker
A: District dropdown off-screen	Major	P2	Fix
C: SMS delayed up to 4 minutes	Medium	P2	

Bug B: Critical / P1

- **Reasoning:** This is data corruption. It breaks AC2, which promises reliable offline sync. Duplicates distort attendance and participant counts, so the data can no longer be trusted. The failure is silent, because nobody sees the duplicate until reports look wrong. There is no workaround, and it affects minors' records.
- **Extra action:** Fix the root cause with an idempotency key on the queued record. Also clean up any duplicates already created in test or live data.

Bug A: Major / P2

- **Reasoning:** The district is a required field in AC1. On affected devices a mentor cannot select it, so registration is blocked on those devices. The severity is Major rather than Critical because only some devices are affected, nothing is corrupted, and registration still works on other devices.
- **Would change if:** Most mentors use low-end phones. Then it becomes Critical / P1, since it blocks the core workflow for most of the user base. Check device analytics or ask the mentors.

Bug C: Medium / P2

- **Reasoning:** AC3 says "immediate", so a 4-minute delay fails the criterion. The severity is Medium because the SMS does arrive and the ID is not lost. The function works but is too slow. The priority is still P2 because the delay has a knock-on effect. A youth with no ID may re-dial and create duplicates, which feeds Bug B.
- **Likely cause:** It may be the SMS gateway rather than our code, so first check provider logs and queueing.
- **Release decision:** Either fix it, or get the Product Owner to formally accept a defined delivery time, for example under 30 seconds.

CI/CD Pipeline & E2E Tooling Selection

1. Framework Trade-off Analysis

I considered Cypress and Playwright for this scenario. Cypress would be attractive if the main priority was getting the team comfortable with automation quickly because its interactive test runner and debugging experience make it relatively straightforward to understand what has failed. My concern, however, would be looking beyond initial adoption to how the test suite will behave as Educate!'s applications and automated coverage grow. For that reason, I would lean towards Playwright.

For Educate!, I would use **Playwright** primarily because I can cover desktop and mobile-responsive journeys within the same framework and run tests across different browser environments. I would also keep the PR pipeline intentionally small, focusing on high-risk user journeys rather than running the entire regression suite every time somebody raises a PR. This gives the development team faster feedback while still protecting main. Broader browser and regression coverage could then run nightly or as part of release testing. This approach gives me a reasonable balance between coverage, execution speed and maintenance effort.

My selection would not be based only on which tool has more features. I would also consider the team's existing programming language capability, because an automation framework only adds value if the team can maintain the tests after implementation.

2. CI/CD Configuration

```
name: Playwright Tests E2E Tests
on:
  pull_request:
    branches: [ main, master ]
jobs:
  test:
    timeout-minutes: 60
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: lts/*
    - name: Install dependencies
      run: npm ci
```

```
  - name: Run Playwright API Tests
    run: npx playwright test
  - uses: actions/upload-artifact@v4
    if: ${{ !cancelled() }}
    with:
      name: playwright-report
      path: playwright-report/
      retention-days: 30
```

Explanation / Thought process

timeout-minutes: 15

I added a timeout because I would not want a failed or hanging E2E test to consume CI resources indefinitely or leave developers waiting unnecessarily for feedback.

I also used:

playwright install --with-deps chromium

instead of installing every Playwright browser.

For pull requests, my priority would be giving developers fast feedback, so I would initially run the critical E2E suite against Chromium. I would then run the broader cross-browser regression suite separately, for example nightly or before a release.

Quality ←→ Speed of developer feedback

The YAML doesn't actually need a special piece of code saying "if tests fail, block the merge.

If this command:

run: npx playwright test --project=chromium

returns a failure, the GitHub Actions job fails.

Then, at repository level, I would make playwright E2E Quality Gate a required status check for the main branch.

That's what makes it a proper merge quality gate.

Quality Gate: If any Playwright test fails, the test command returns a non-zero exit status and the GitHub Actions job fails. I would configure the main branch so that this job is a required status check. This means a developer cannot merge the PR until the failed automated tests have been addressed.